

Manual de Usuario
Análisis y Diseño De Algoritmos

Presentado a:

Luz Enith Guerrero Mendieta

Realizado por:

Leidy Johana Nieto
Jerónimo García Parra



Universidad de Caldas
2026-1

Índice

1. Finalidad.....	3
2. Herramientas Necesarias.....	3
3. Instalación de Herramientas.....	5
4. Configuración del Proyecto.....	18
5. Ejecución del Proyecto.....	22
6. Entendimiento de Resultados.....	24
7. Bibliografía.....	29

1. Finalidad

La finalidad de este manual, es que un usuario que tenga dificultades para la ejecución del proyecto pueda guiarse para obtener respuestas y soluciones en caso de que sufra percances.

Este manual tiene toda la información relacionada a la instalación de herramientas necesarias para ejecutar el proyecto, como se instalan, las versiones necesarias y el paso a paso de como lograr la ejecución correcta del algoritmo creado y entender qué resultados deben entregar y cómo se obtienen.

2. Herramientas Necesarias

Para comenzar con el manual tendremos que definir las herramientas necesarias para la ejecución adecuada de nuestro proyecto.

Primero necesitamos un editor de texto que nos permita ejecutar el repositorio del proyecto

Una de las opciones más interesantes y más fáciles de utilizar es **Visual Studio Code**.

Visual Studio Code es un potente editor de código fuente ligero, gratuito y multiplataforma desarrollado por Microsoft, diseñado para optimizar el flujo de trabajo de los programadores modernos. Su característica más destacada es su altísima capacidad de personalización a través de un inmenso mercado de extensiones, lo que permite adaptarlo para escribir en prácticamente cualquier lenguaje de programación. Además de ser un simple editor de texto, VS Code integra de manera nativa herramientas esenciales para el desarrollo profesional, como una consola de comandos, control de versiones con Git, autocompletado inteligente de código y capacidades avanzadas de depuración de errores, convirtiéndolo en el entorno de desarrollo preferido y estándar en la industria tecnológica actual.

También será necesario un compilador para que la ejecución de nuestro proyecto ocurra sin ningún inconveniente, está lenguaje de programación utilizado fue **Python**.

Python es un lenguaje de programación de alto nivel, interpretado y de propósito general, reconocido mundialmente por su sintaxis clara y legible que prioriza la simplicidad y la rapidez de desarrollo. En el contexto científico y de ingeniería de software, destaca por su inmenso ecosistema de bibliotecas especializadas (como NumPy o pandas) que facilitan enormemente el manejo de estructuras de datos complejas, matrices de probabilidad y cálculos matemáticos avanzados. Gracias a su versatilidad, su compatibilidad multiplataforma y su ejecución dinámica, Python se ha consolidado como la herramienta estándar en la industria para el análisis de datos, la optimización de algoritmos y el modelado de sistemas complejos, permitiendo a los investigadores enfocarse en la lógica del problema en lugar de en las minucias del código.

Para el manejo adecuado de la consola y la copia del repositorio en nuestra entorno virtual haremos uso de la herramienta de **Git**.

Git es un sistema automatizado de control de versiones distribuido, gratuito y de código abierto, diseñado para rastrear milimétricamente los cambios en el código fuente durante el ciclo de vida del desarrollo de software. Su arquitectura descentralizada permite que cada programador posea una copia local completa de todo el historial del proyecto, lo que garantiza una enorme velocidad de ejecución y seguridad contra la pérdida de datos. En el contexto de la ingeniería y la investigación, su mayor fortaleza radica en su poderoso sistema de ramificaciones (branching), el cual permite crear entornos aislados para experimentar con nuevas características o resolver errores de forma colaborativa sin poner en riesgo la versión principal y estable del sistema. Hoy en día, junto con plataformas que lo alojan como GitHub o GitLab, Git es el estándar absoluto de la industria para asegurar la trazabilidad, el trabajo en equipo y la integridad en proyectos tecnológicos complejos.

Git nos facilitara la vida mediante el uso de la página de Github, para la copia del repositorio y el control de versiones del proyecto, en caso de querer mejorarlo o proponer cambios para más adelante.

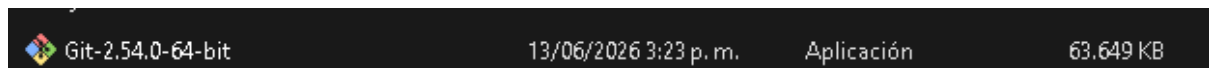
Nota: Si llegamos a presentar problemas, en la ejecución del proyecto, hay un motivo principal y es que hay algunas herramientas de python que necesitan ser compiladas por otros lenguajes, como por ejemplo C++, en caso dado de que esté ocurriendo este problema específicamente tendremos que instalar adicionalmente una herramienta llamada Visual Studio Tools, esta nos dará la solución a la compilación de C++ y nos brinda la facilidad para evitar tener problemas de ejecución.

3. Instalación de Herramientas

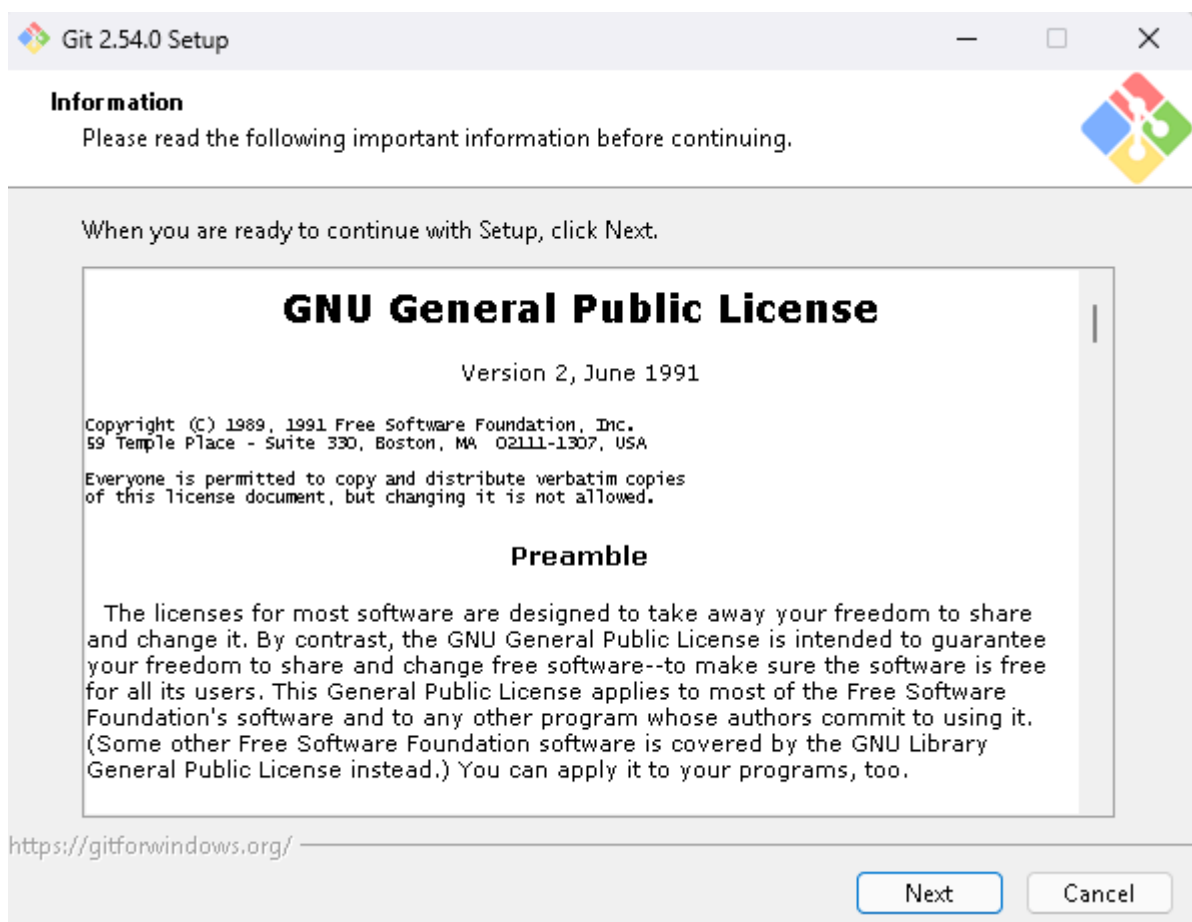
Empecemos por Git, esta herramienta es la primera que debemos tener instalada en nuestro sistema para poder tener la facilidad con su consola Git Bash y más compatibilidad con la página web de Github.

Vamos a el link señalado de la página oficial de la herramienta. [Git - Install for Windows](#)

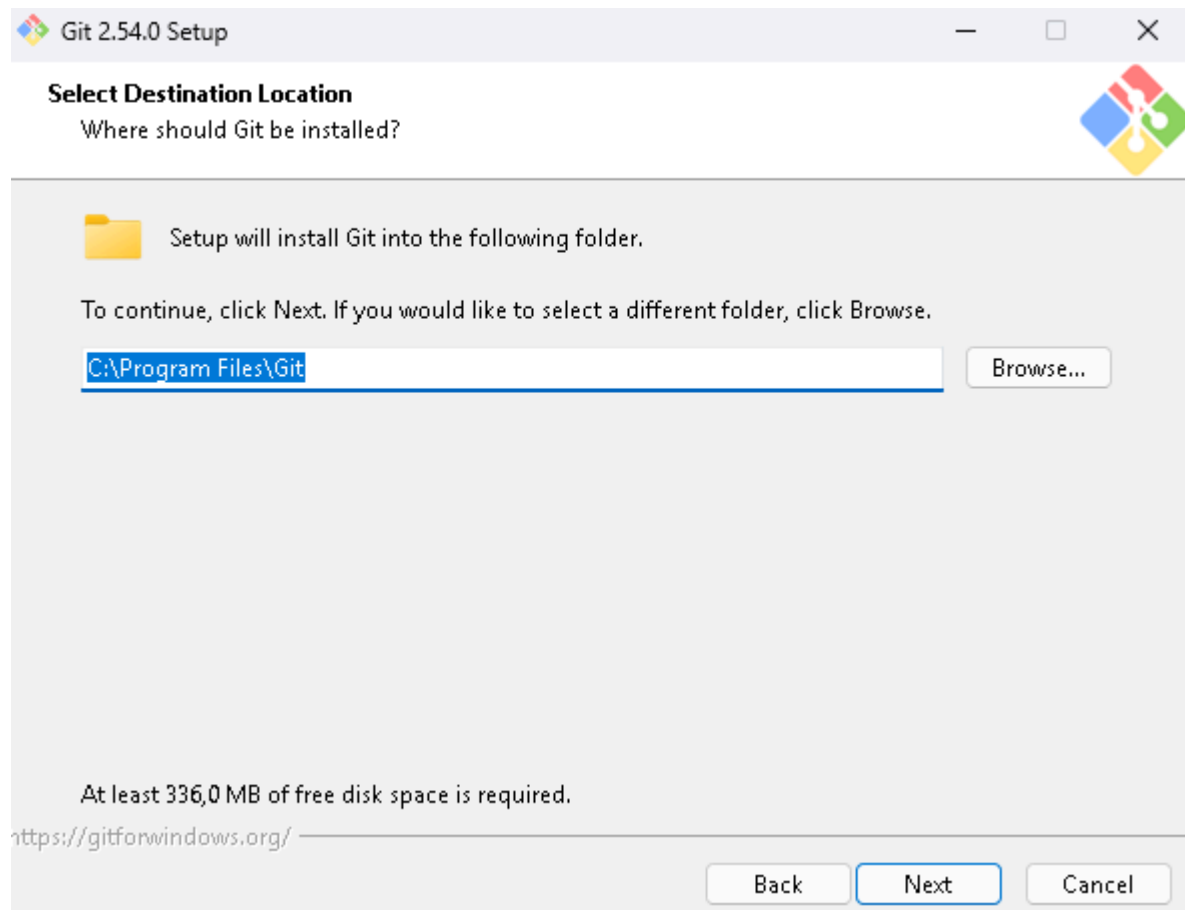
Después nos dirigimos al archivo que nos deja:



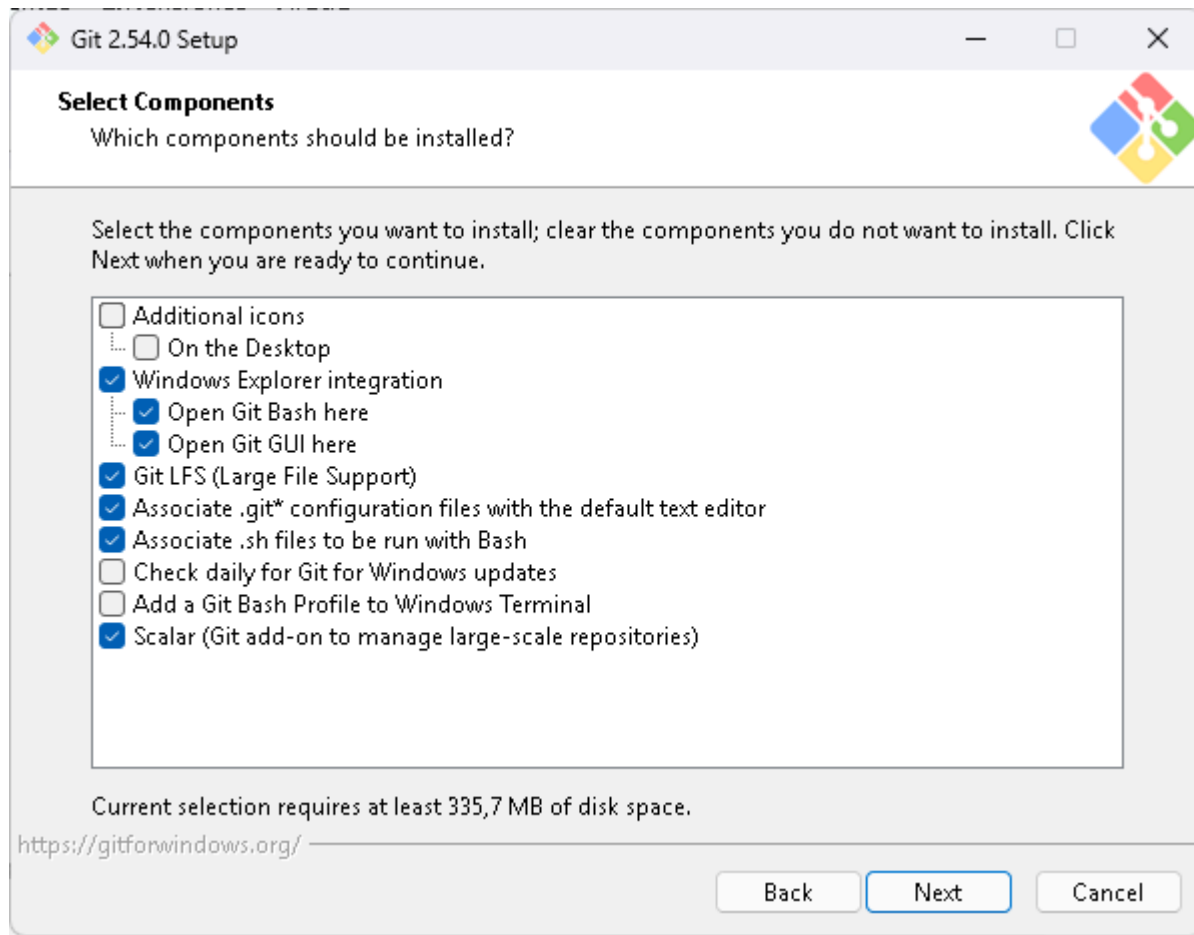
Lo ejecutamos y seguimos los pasos a continuación:



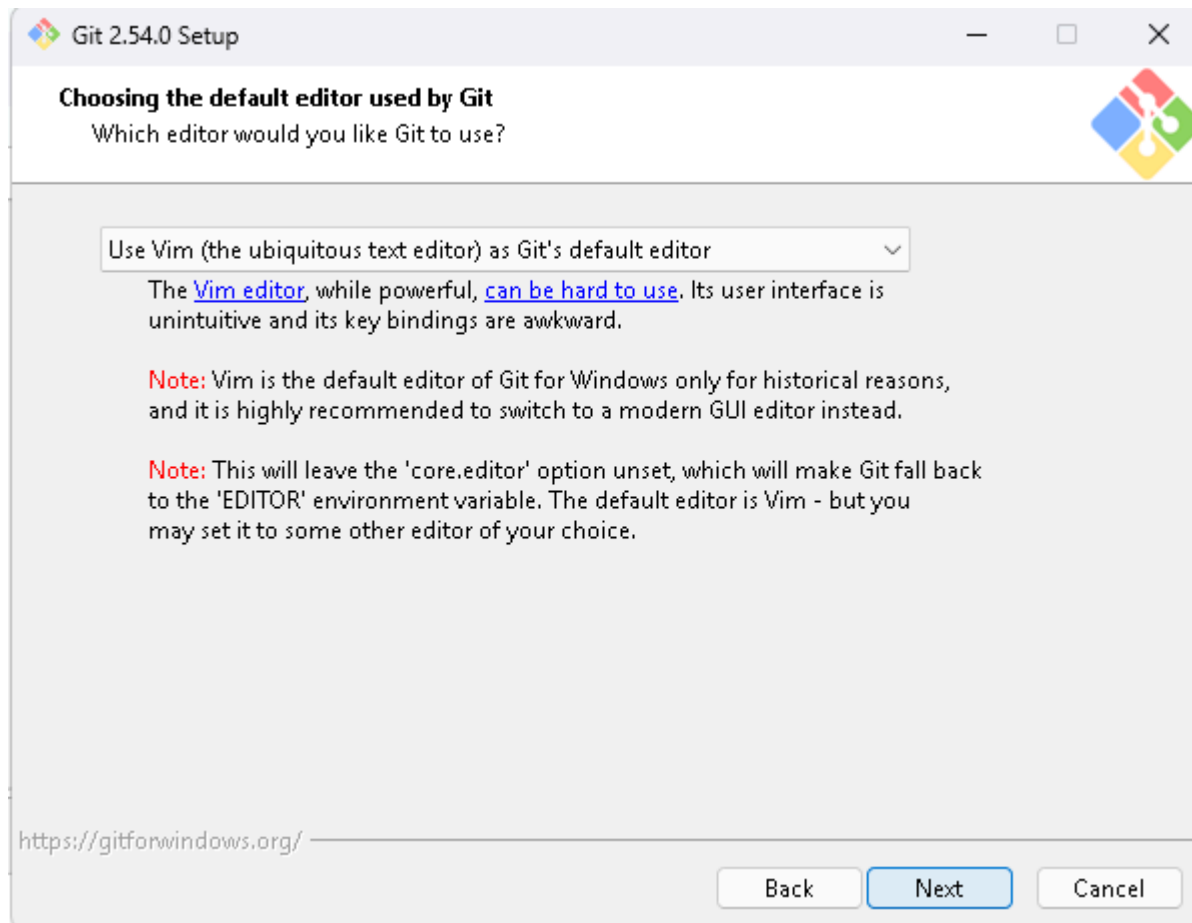
Le damos a Next.



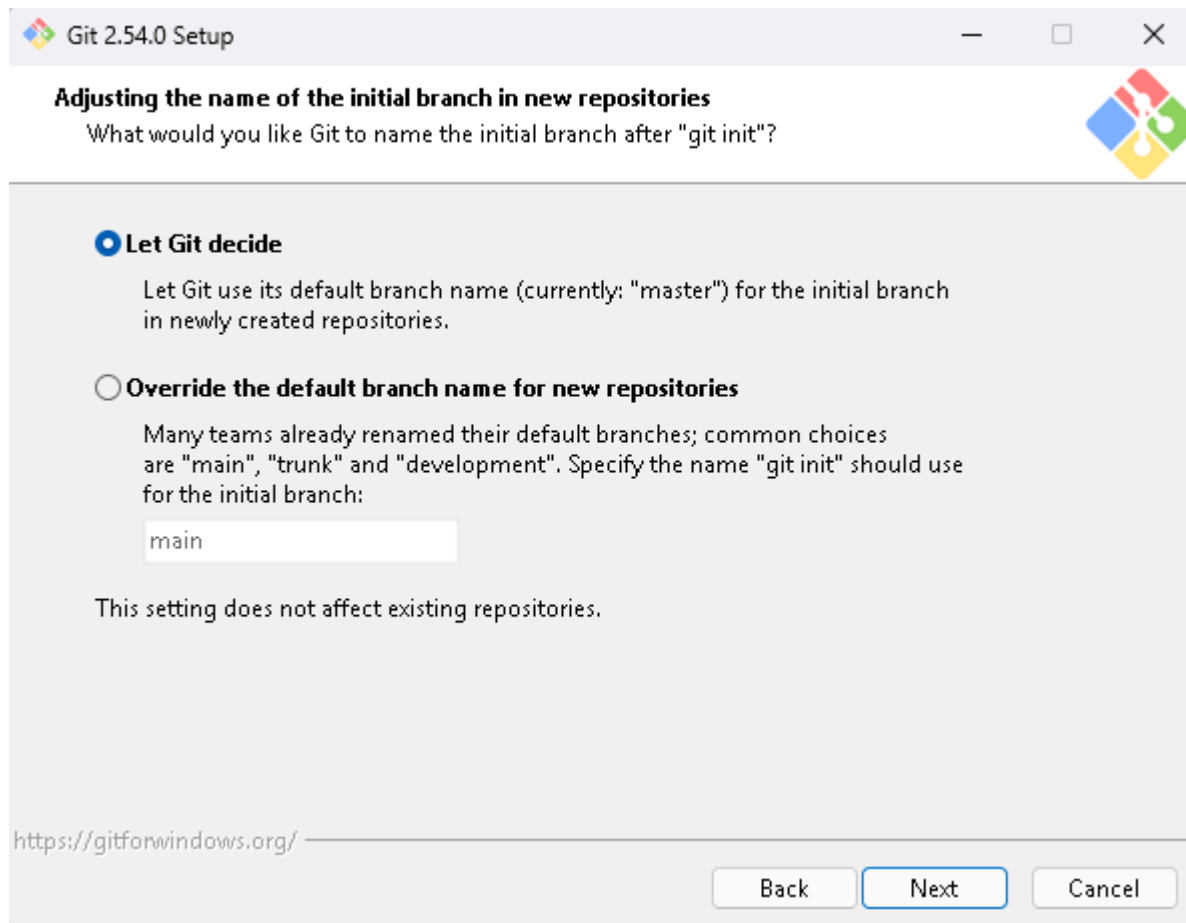
Si queremos podemos cambiar la ubicación del Git si no damos a Next.



Dejamos como está todo por defecto y le damos a Next.

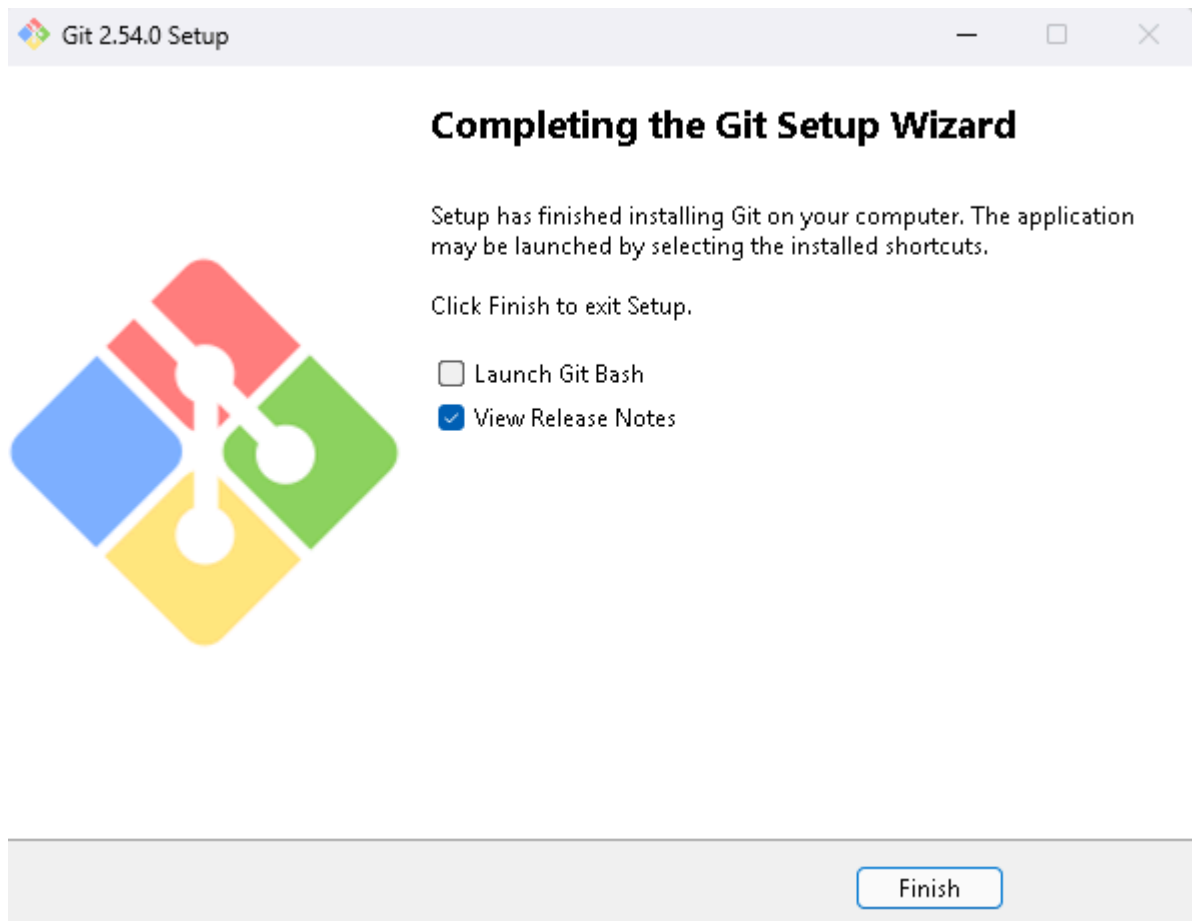


Dejamos vim y le damos a Next.



Dejamos la primera opción y le damos a Next.

Seguimos las opciones por defecto y le damos a next hasta que nos salga la opción de install. Esperamos hasta que finalice.

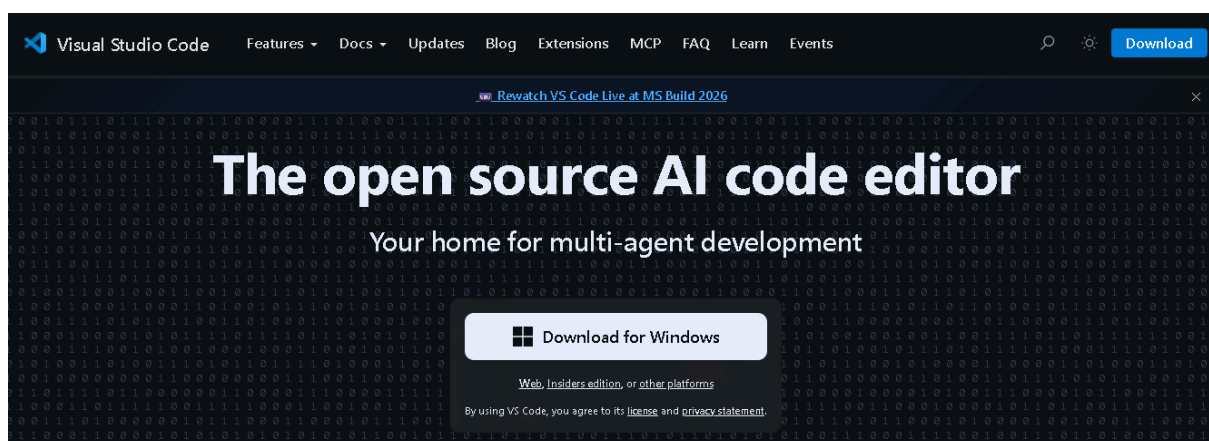


Desmarcamos la segunda opción y le damos a finalizar.

Ahora pasamos a la instalación de la herramienta de Visual Studio Code

Entramos primero al link de la página oficial: [Visual Studio Code - The open source AI code editor | Your home for multi-agent development](https://code.visualstudio.com/)

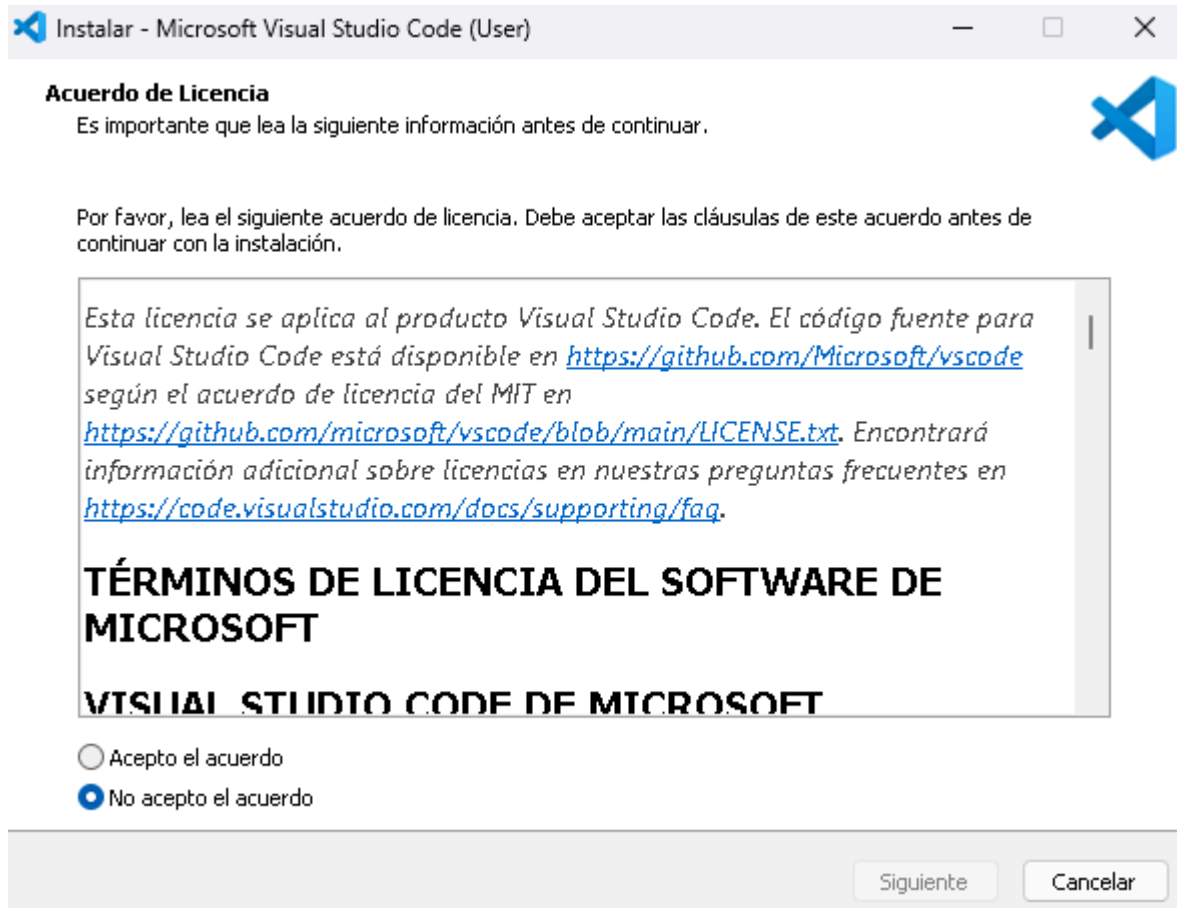
Después le damos a Download For Windows, o si tienen un sistema operativo diferente, les aparecerá descargar para el SO que tengan instalado.



Esperamos a que la descarga finalice y le ubicamos en la carpeta donde se suele almacenar los archivos que serían las descargas.

 VSCodeUserSetup-x64-1.124.2	13/06/2026 3:30 p. m.	Aplicación	180.644 KB
---	-----------------------	------------	------------

Abrimos el ejecutable y seguimos los pasos:



Aceptamos el acuerdo de licencia.

Instalar - Microsoft Visual Studio Code (User)

Seleccione las Tareas Adicionales

¿Qué tareas adicionales deben realizarse?



Seleccione las tareas adicionales que desea que se realicen durante la instalación de Visual Studio Code y haga clic en Siguiente.

Accesos directos adicionales:

☐ Crear un acceso directo en el escritorio

Otros:

☐ Agregar la acción "Abrir con Code" al menú contextual de archivo del Explorador de Windows

☐ Agregar la acción "Abrir con Code" al menú contextual de directorio del Explorador de Windows

☒ Registrar Code como editor para tipos de archivo admitidos

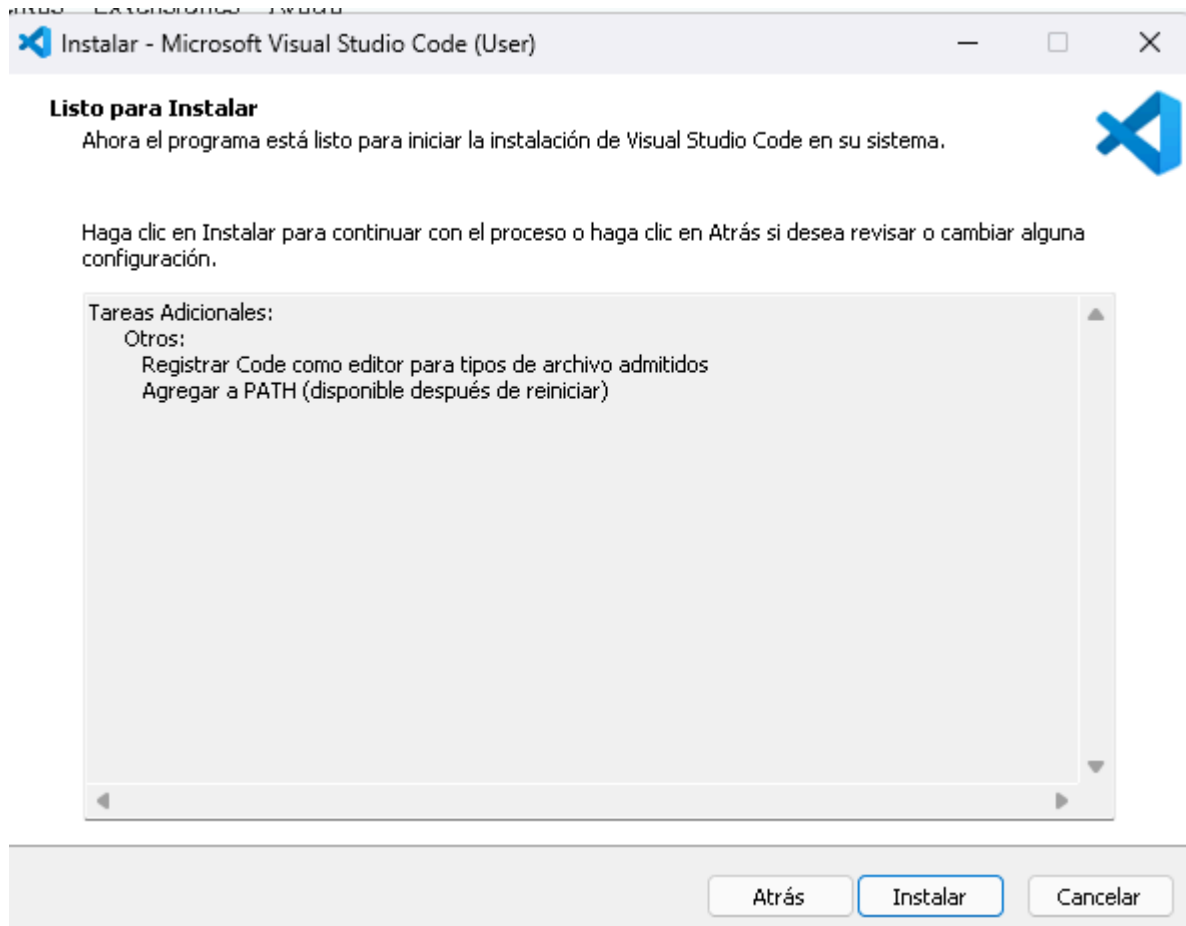
☒ Agregar a PATH (disponible después de reiniciar)

Atrás

Siguiente

Cancelar

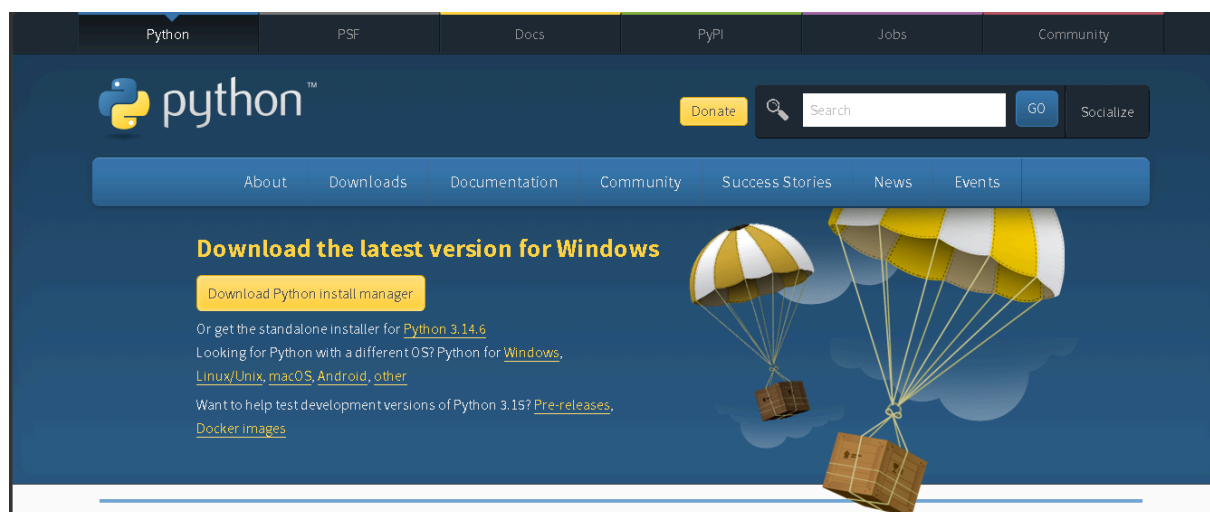
Aseguramos la palomita a la opción de agregar al Path y si queremos también a la de crear un acceso directo en el escritorio para tener la aplicación a la mano.



Finalmente le damos a instalar y debemos esperar hasta que se termine la instalación. Cuando esta finalice le damos a terminar o finalizar, y si podemos hacemos un reinicio a nuestra computadora.

Vamos ahora con la herramienta de Python.

Nos dirigimos al siguiente enlace: [Download Python | Python.org](https://www.python.org/downloads/)



Estando aquí podemos instalar la versión 3.14.6, que es la última versión, en nuestro proyecto esta última versión sirve para ejecutar el proyecto en la estrategia GeoMIP, pero podría generar errores en la versión de Qnodes por lo que por recomendación propia recomiendo la versión de Python 3.12. Esta versión tiene todas las compatibilidades necesarias para la ejecución normal del proyecto, si nos decidimos de instalar la 3.12, seguimos bajando en la página hasta que nos encontremos con:

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.13.10	Dec. 2, 2025	 Download	Release notes
Python 3.13.9	Oct. 14, 2025	 Download	Release notes
Python 3.13.8	Oct. 7, 2025	 Download	Release notes
Python 3.13.7	Aug. 14, 2025	 Download	Release notes
Python 3.13.6	Aug. 6, 2025	 Download	Release notes
Python 3.13.5	June 11, 2025	 Download	Release notes
Python 3.13.4	June 3, 2025	 Download	Release notes
Python 3.13.3	April 8, 2025	 Download	Release notes

[View older releases](#)

Aquí bajamos la lista hasta que encontremos la versión 3.12.10, ya que esta es la que brinda el instalador, le damos clic a Download.

Python 3.12.10

Note: Python 3.12.10 has been superseded by [Python 3.12.13](#).

Release date: April 8, 2025

This is the tenth maintenance release of Python 3.12

Python 3.12.10 is the latest maintenance release of Python 3.12, and the last full maintenance release. Subsequent releases of 3.12 will be *security-fixes only*. This last maintenance release contains about 230 bug fixes, build improvements and documentation changes since 3.12.9.

Entramos a esta pagina, bajamos hasta el fondo hasta que nos encontremos.

Files

macOS

Download macOS 64-bit universal2 installer

Windows

Download Python install manager

Source release

Download XZ compressed source tarball

Version	Operating system	Description	File size	Sigstore	SBOM	GPG	MD5 checksum
Gzipped source tarball	Source release		25.9 MB	.sigstore	SPDX	SIG	35c03f014408e26e2b06d576c19cac54
XZ compressed source tarball	Source release		19.6 MB	.sigstore	SPDX	SIG	0c0a6cce86d7611aed691d61e0599de0
macOS 64-bit universal2 installer	macOS	for macOS 10.13 and later	43.6 MB	.sigstore		SIG	8f4989592c9412e51fcad70273442c22
Windows installer (64-bit)	Windows	Recommended	25.7 MB	.sigstore	SPDX	SIG	5eddb0b6f12c852725de071ae681dde4
Windows installer (32-bit)	Windows		24.5 MB	.sigstore	SPDX	SIG	ff3e7e2582811ade44916d9c3d2a64b2
Windows installer (ARM64)	Windows	Experimental	25.0 MB	.sigstore	SPDX	SIG	37df52717faa6c514342b8d43cae7026
Windows embeddable package (64-bit)	Windows		10.6 MB	.sigstore	SPDX	SIG	fe8ef205f2e9c3ba44d0cf9954e1abd3
Windows embeddable package (32-bit)	Windows		9.5 MB	.sigstore	SPDX	SIG	93beb9042be23d7b5cd528fa15e8fea7

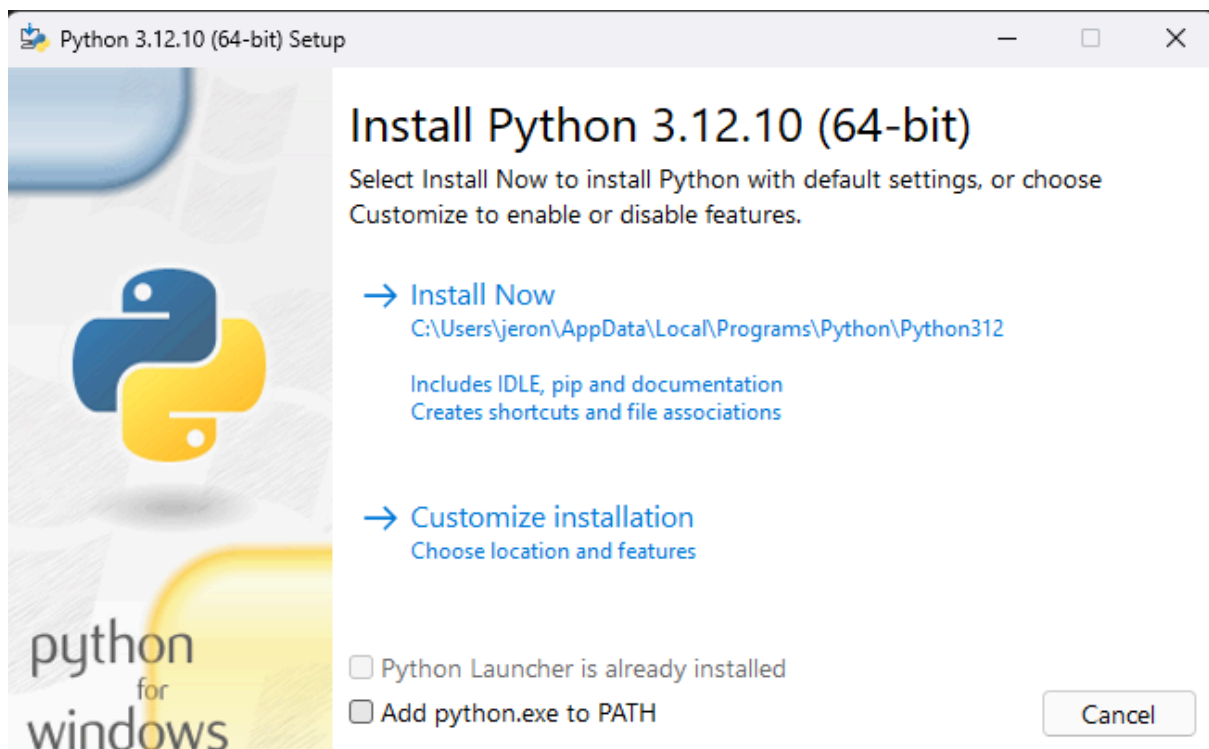
De ahí le damos a Windows Installer (64 Bits) en Windows.

Ahí comenzará la descarga y deberemos esperar a que finalice.

Después ubicamos el archivo que nos dejara descargado en nuestra máquina

 python-3.12.10-amd64	13/06/2026 3:41 p. m.	Aplicación	26.333 KB
--	-----------------------	------------	-----------

Lo ejecutamos y se abrirá la siguiente ventana:



Debemos elegir Add python.exe to PATH y también el Python Launcher por si acaso y le damos a Install Now y empezará a instalarse, demora un rato hasta que deje, por ultimo saldra una opción para deshabilitar una opción de un rango máximo, si queremos la podemos marcar o dejar asi y darle a finalizar.

En caso de presentar el problema mencionado con las configuraciones del C++ con sus compiladores, nos dirigimos al siguiente link: [Descargas de Visual Studio y VS Code para Windows, Mac, Linux](#)

Descargas



Visual Studio 2026

El IDE más completo para desarrolladores de .NET y C++ en Windows para crear aplicaciones web, en la nube, de escritorio, móviles, servicios y juegos.

Insiders

La compilación Visual Studio 2026 Insiders ya está aquí.

[Descargar](#)[Más información →](#)

Community

IDE eficaz, gratuito para estudiantes,

Professional

IDE profesional más adecuado para

Enterprise

Escalable, solución integral para equipos

Nos mostrará esta ventana, bajamos hasta que nos encontremos con las siguientes opciones.

Todas las descargas

[Expandir todo →](#) [Contraer todo →](#)

> Visual Studio

✓ Herramientas para Visual Studio

Herramientas remotas para Visual Studio 2026

Herramientas remotas para Visual Studio 2022 permite la implementación de aplicaciones, la depuración y la comprobación remotas, la generación de perfiles de rendimiento y la ejecución de pruebas unitarias en equipos que no tengan instalado Visual Studio. El uso de esta herramienta requiere una licencia válida de Visual Studio.

[Descargar](#)

Y buscamos las que dicen herramientas de compilación para visual studio.

Herramientas de compilación para Visual Studio 2026

Estas herramientas de compilación permiten compilar proyectos de Visual Studio desde una interfaz de línea de comandos. Algunos proyectos admitidos son: ASP.NET, Azure, escritorio de C++, ClickOnce, contenedores, .NET Core, .NET Desktop, Node.js, Office y SharePoint, Python, TypeScript, pruebas unitarias, UWP, WCF y Xamarin. El uso de esta herramienta requiere una licencia de Visual Studio válida, a menos que esté creando dependencias de código abierto para su proyecto. Consulte la [licencia Build Tools](#) para obtener más información.

[Descargar](#)

¿Está buscando una de las [líneas base de mantenimiento a largo plazo \(LTSC\)](#) de Visual Studio 2022? Puede encontrarlas [aquí](#).

Y le damos a descargar.



vs_BuildTools

13/06/2026 3:52 p. m.

Aplicación

5.564 KB

Buscamos el archivo que nos dejó y lo ejecutamos.

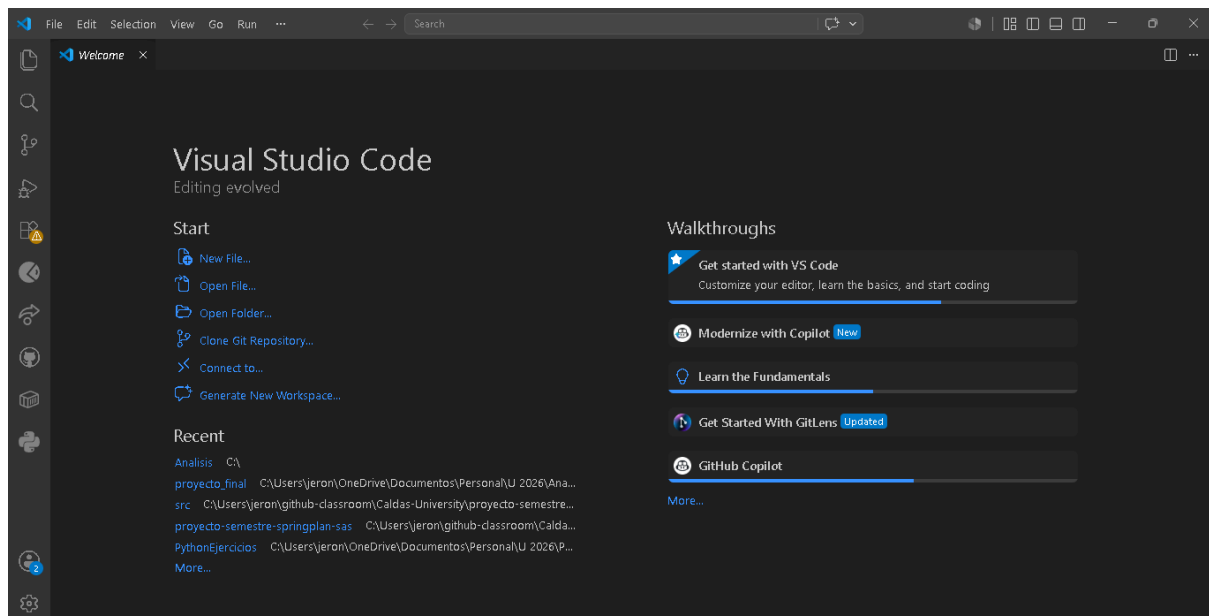
Va a demorar un poco pero cuando finalice nos dejará una pestaña como estas.



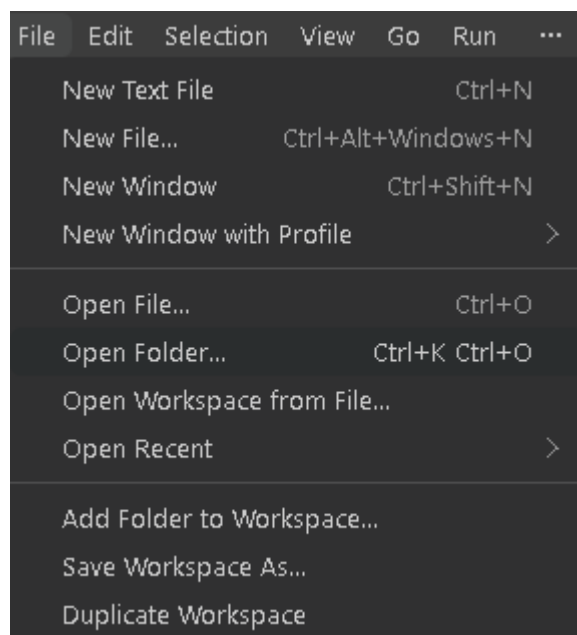
Elegimos la primera opción de Desarrollo para el escritorio en C++ y le damos a instalar, se demora un poco, mientras instala cuando acabe le damos a finalizar, y reiniciamos nuestro computador, ya con esto tendríamos todo lo necesario para instalar y ejecutar nuestro proyecto de análisis de las particiones de un sistema dado.

4. Configuración del Proyecto

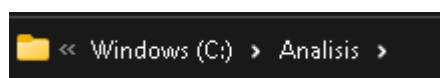
Una vez tengamos todas nuestras herramientas procedemos a abrir el Visual Studio Code.



Tendremos esta interfaz, le daremos a File en la esquina superior izquierda, y nos abrirá un montón de opciones.



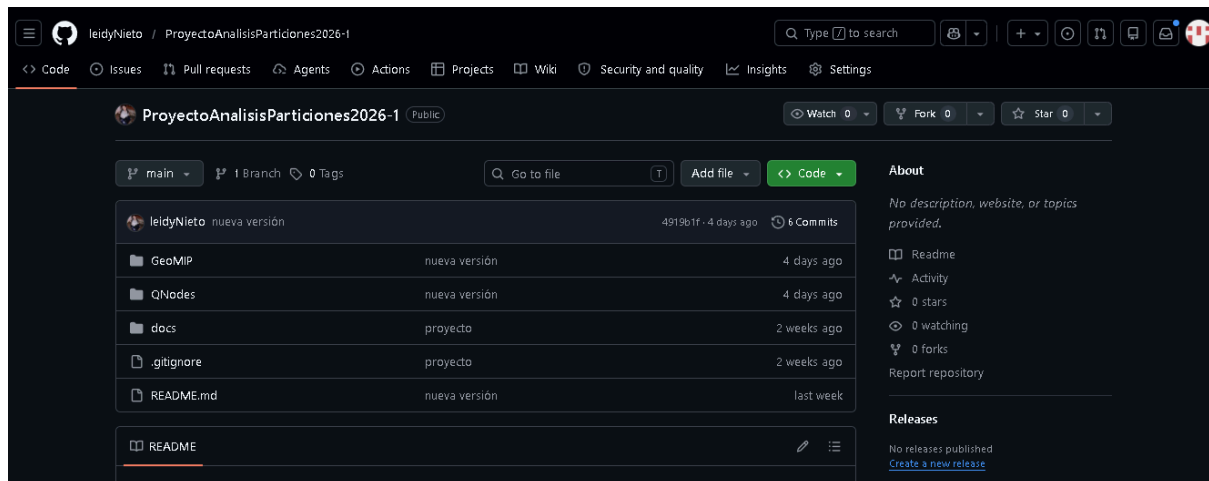
Le damos a open folder, y crearemos una carpeta donde haremos la clonación de nuestro repositorio.



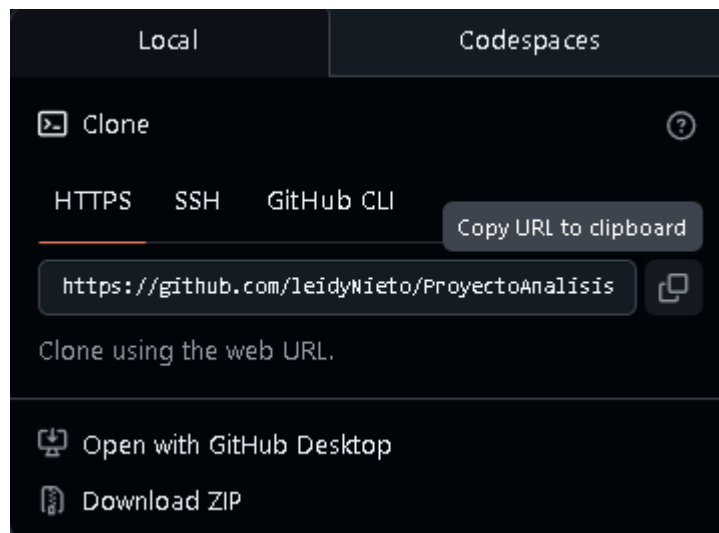
En mi caso crea una carpeta llamada análisis en el disco local C, ahora después de creada, nos mostrará un aviso de si confiamos en donde vamos a ubicarnos y le damos que si confiamos en la ubicación, ahora nos saldrá una ventana que ya nos ubica en la carpeta.

Ahora nos vamos a dirigir al siguiente enlace de GitHub que ahí es donde tenemos el proyecto.

[leidyNieto/ProyectoAnálisisParticiones2026-1](https://github.com/leidyNieto/ProyectoAnálisisParticiones2026-1)

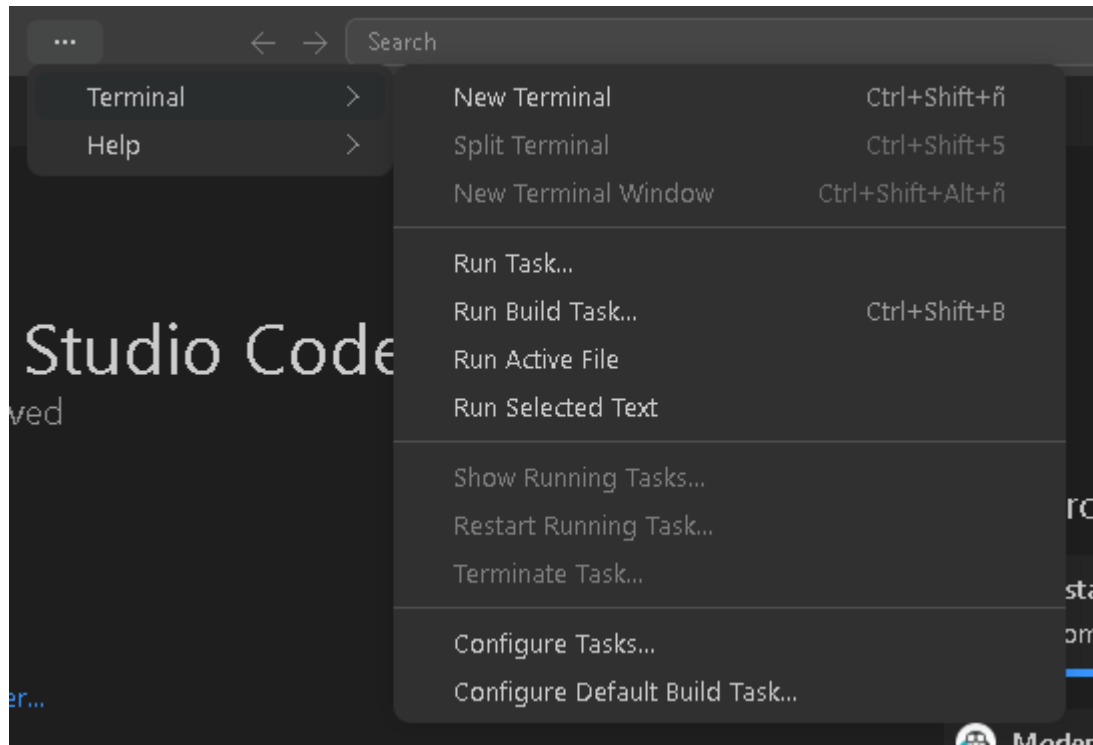


Ubicados aquí, le damos al botón verde que nos dice Code.

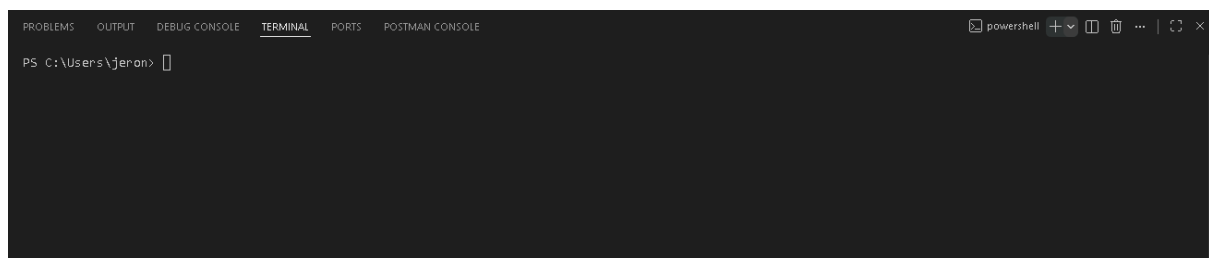


Aquí le damos al botón al lado del link, y le damos a copiar a nuestro portapapeles.

De ahí nos dirigimos nuevamente al Visual.



Le damos a los tres puntos en la sección de arriba y le damos a terminal y **New Terminal**.



Se nos abrirá una consola así. Le damos a la flecha apuntando hacia abajo, al lado del signo más y le damos a la opción de Git Bash

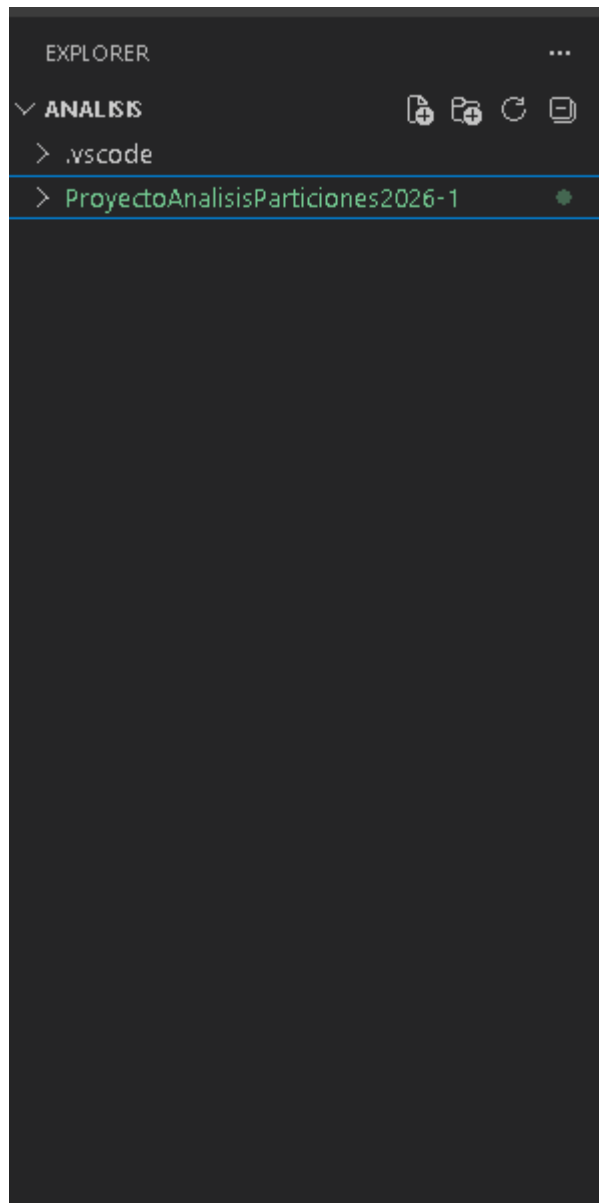


Nos dejará esta interfaz y pondremos el comando **git clone** (*link que copiamos al portapapeles*), y finalmente le damos enter.

```
jeron@LPBlaz MINGW64 ~  
$ git clone https://github.com/leidyNieto/ProyectoAnalisisParticiones2026-1.git
```

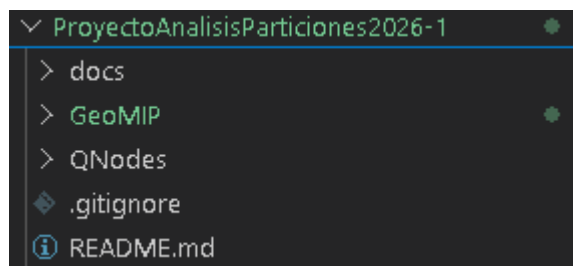
Ahí empezará a clonar el proyecto.

Finalmente nos dejará esta carpeta ya a disposición de poder ejecutar el proyecto.



5. Ejecución del Proyecto

Primero debemos entender que tenemos dos proyectos en el mismo repositorio.



Cuando abrimos la carpeta principal tenemos GeoMIP Y QNodes, ambos se pueden ejecutar mediante su archivo Exec.py, para ejecutar los proyectos debemos hacer uso de la consola, por lo que primero creamos una terminal nueva.

```
jeron@LPBlaz MINGW64 /c/Análisis
$ █ Type copilot to use Copilot CLI.
    Start typing to dismiss or don't show this again.
```

Estando aquí empezaremos a entrar en las carpetas del proyecto mediante el comando **cd**. Podemos usar el Tab para empezar a autocompletar las rutas de las carpetas.

```
jeron@LPBlaz MINGW64 /c/Análisis
> $ cd ProyectoAnálisisParticiones2026-1/█
```

Después volvemos a usar **cd** para entrar a GeoMIP o QNodes, vamos primero con QNodes.

```
jeron@LPBlaz MINGW64 /c/Análisis/ProyectoAnálisisParticiones2026-1 (main)
$ cd QNodes/

jeron@LPBlaz MINGW64 /c/Análisis/ProyectoAnálisisParticiones2026-1/QNodes (main)
$ █
```

Ubicados en QNodes tenemos que instalar primero un entorno virtual llamado uv, esto es lo que nos permite ejecutar el proyecto de forma segura, ponemos ahí mismo ***pip install uv***

```
jeron@LPBlaz MINGW64 /c/Análisis/ProyectoAnálisisParticiones2026-1/QNodes (main)
$ pip install uv█
```

Empezará a instalar el entorno virtual, cuando acabe en QNodes nos quedará la siguiente carpeta llamada **.venv**, que aparecerá como medio desvanecida, esto es porque es una carpeta oculta del sistema.

```
▼ QNodes
  > .docs
  > .logs
  > .venv
```

Después de hacer eso, tendremos que poner otro comando en nuestra consola, que sería haciendo uso del entorno virtual que es **uv sync**.

```
jeron@LPBlaz MINGW64 /c/Analisis/ProyectoAnalisisParticiones2026-1/QNodes (main)
$ uv sync
```

Se procederá a instalar todas las dependencias necesarias para ejecutar el proyecto con normalidad, cuando finalice nos mostrará un listado de todas las dependencias que se instalaron en el sistema.

Ya con eso habremos instalado todo lo necesario para Qnodes ahora vamos para el GeoMIP, podemos abrir una nueva terminal y ubicarnos en esa carpeta.

```
jeron@LPBlaz MINGW64 /c/Analisis/ProyectoAnalisisParticiones2026-1/GeoMIP (main)
$
```

Una vez ubicados aquí, debemos entrar primero a la carpeta Src y después a la carpeta general de GeoMIP donde ahí si ya pondremos los comandos necesarios.

```
jeron@LPBlaz MINGW64 /c/Analisis/ProyectoAnalisisParticiones2026-1/GeoMIP (main)
• $ cd src/

jeron@LPBlaz MINGW64 /c/Analisis/ProyectoAnalisisParticiones2026-1/GeoMIP/src (main)
• $ cd Method2_Dynamic_Programming_Reformulation/

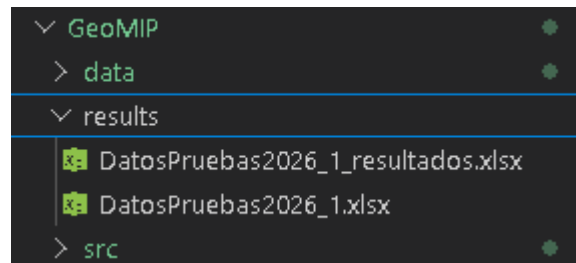
jeron@LPBlaz MINGW64 /c/Analisis/ProyectoAnalisisParticiones2026-1/GeoMIP/src/Method2_Dynamic_Programming_Reformulation (main)
$
```

Una vez nos ubiquemos aquí, ahora sí pondremos el comando de **uv sync**, ya no tendremos que hacer el **pip install uv**, ya que este ya está instalado en nuestro sistema, entonces ya de por si va a crear la carpeta .venv en el GeoMIP, ya podremos ejecutar el proyecto con un comando que mostraré a continuación.

6. Entendimiento de Resultados

Para ejecutar el proyecto usamos el comando ***uv run exec.py***

Pero antes de eso vamos a entender que es lo que va hacer el programa al ejecutarse.



Primero va a encontrar una carpeta llamada Results dónde estarán los casos de prueba que se van a usar, en este caso es un excel que nos muestra lo siguiente.

	A	B	C	D	E	F	G
1	Estado inicial	10000000000000000000000000000000					
2	Sistema:	ABCDEFGHIJKLMNOPQRSTUVWXYZ					
3	Sistema Candid	ABCDEFGHIJKLMNOPQRSTUVWXYZ					
4		Subsistema			QNodes		
5	#Prueba	Alcance o Purview (t+1)	Mecanismo(t)	Partición	Pérdida	Tiempo	Part
6	1	ABCDEFGHIJKLMNOPQRSTUVWXYZ	ABCDEFGHIJKLMNOPQRSTUVWXYZ				
7	2	ABCDEFGHIJKLMNOPQRSTUVWXYZ	ABCDEFGHIJKLMNOPQRSTUVWXYZ				
8	3	ABCDEFGHIJKLMNOPQRSTUVWXYZ	BCDEFGHIJKLMNOPQRSTUVWXYZ				
9	4	ABCDEFGHIJKLMNOPQRSTUVWXYZ	BCDEFGHIJKLMNOPQRSTUVWXYZ				

Es una hoja donde nos darán el estado inicial, el sistema y el sistema candidato, y 50 pruebas por hoja, donde se evaluarán la cantidad de nodos según sea la hoja, si son 10 nodos, 15 o 25, cada hoja tiene sus 50 pruebas aparte.

Volviendo a la consola, cuando ejecutamos usemos el caso de GeoMIP.

```
=== Hoja: 10A-Elementos | TPM: N10A.csv ===
Prueba #1 | alcance=1111111111 mecanismo=1111111111
CRITICAL (16:42:51): Original creado.
CRITICAL (16:42:51): Original:
CRITICAL (16:42:51): Candidato creado.
CRITICAL (16:42:51): Subsistema creado.
CRITICAL (16:42:51): empieza.
```


Cuando se ejecuta, primero encuentra la Hoja de cálculo, donde hará los cálculos para hallar esa bipartición. identifica el alcance, el mecanismo, crea el candidato y el subsistema y empieza a evaluar.

```
K=2
| I || A,B,C,D,E,F,G,H,I |
| a || b,c,d,e,f,g,h,i,j |
0.7412739

K=3
| H,I || A,B,C,D || E,F,G,I |
| a,b || c,d,e,f || g,h,i,j |
1.4366798

K=4
| A,H,I || B,C,D || E,F || G,I |
| b,f || h,i,j || a,c || d,e,g |
1.4039364

K=5
| A,H,I || E,F || G,I || B || C,D |
| b,f || a,c || d,e,g || h || i,j |
1.3069613

-> Mejor k=2 perdida=0.7413 tiempo=189.720s
```

Cuando empieza a solucionar, nos va mostrando la mejor partición que puede hacer según la cantidad de veces que se puede partir, si se hace una bipartición o algo mejor, al final nos da la k que generó la menor pérdida y el tiempo tardado, esto lo hará hasta que llegue hasta las 50 pruebas.

Cuando ya cambia de hoja para evaluar unos nodos de mayor tamaño, se almacenan todos los resultados de esa hoja en un excel copia, de la hoja que utilizo, pero llamada con su nombre más el resultado.

Para el caso de Qnodes sería así.

Q-Nodes – Evaluación k-particiones (k = 2 ... 5)

K=2 ◀ MEJOR

```
| A,B,C,D,E,G,H,I,J || F |  
| a,b,c,d,e,f,g,h,i,j || ∅ |  
φ = 0.0340806 tiempo = 0.7970s
```

K=3

```
| A,C,D,E,G,H,I,J || B || F |  
| a,b,c,d,e,f,g,h,i,j || ∅ || ∅ |  
φ = 0.0720924 tiempo = 0.6448s
```

K=4

```
| A || C,D,E,G,H,I,J || B || F |  
| ∅ || a,b,c,d,e,f,g,h,i,j || ∅ || ∅ |  
φ = 0.1377742 tiempo = 0.5624s
```

K=5

```
| A || C,D,E,G,H,I,J || B || F || I |  
| ∅ || a,b,c,d,e,f,g,h,i,j || ∅ || ∅ || ∅ |  
φ = 0.2652038 tiempo = 0.3933s
```

Esto se visualiza así en los excels de los resultados.

Geometric		
Partición	Pérdida	Tiempo
I A,B,C,D,E,F,G,H,I a b,c,d,e,f,g,h,i,j	0,74	1,55
A,B,C,D,G,H,I,I E,F a,b,c,d,e,f,g,h i	0,69	0,41
H A,B,C,D,E,F,G,I,I b c,d,e,f,g,h,i,j	0,68	0,40
C,D,F,G,H,I A,B,E,I d,e,f,i b,c,g,h	0,39	0,11
F A,B,C,D,E,G,H,I,I b,d,e,i a,g,h	0,24	0,04
A,G,H,I B,C,D,E,F,I a,c,e,i g	0,23	0,02
B,F,G,I,I A,C,D,E,H b,f,h,i d	0,19	0,02
I A,B,C,D,E,F,G,H a b,c,d,e,f,g,h,i,j	0,68	1,32
B A,C,D,E,F,G,H,I a b,c,d,e,f,g,h,i	0,49	0,35
A,H B,C,D,E,F,G,I b,c d,e,f,g,h,i,j	0,44	0,38
B,F,H,I A,C,D,E,G c,d,f,h b,e,g,i	0,28	0,10
C,F,G,I A,B,D,E,H a,b,d,e g,h,i	0,31	0,03
A,G,H,I B,C,D,E,F a,c,e,i g	0,14	0,01
B,F,G,I A,C,D,E,H b,f,h,i d	0,18	0,02
I B,C,D,E,F,G,H,I a b,c,d,e,f,g,h,i,j	0,74	1,33
B,C,D,E,G,H,I,I F a,b,c,d,e,f,g,h i	0,53	0,36
H B,C,D,E,F,G,I,I b c,d,e,f,g,h,i,j	0,57	0,37
B,C,D,F,G,H,I,I E b,c,d,e,f,g,h i	0,39	0,11
F B,C,D,E,G,H,I,I b,d,e,i a,g,h	0,20	0,03
G,H,I B,C,D,E,F,I a,c,e,i g	0,22	0,01
B,F,G,I,I C,D,E,H b,f,h,i d	0,13	0,02

Asi mismo funciona para Qnodes. Con eso ya entendemos cómo ejecutar el proyecto y ya quedamos listos para hacer las pruebas que necesitamos.

QNodes		
Partición	Pérdida	Tiempo
A C,D,E,G,H,I,J B F Ø a,b,c,d,e,f,g,h,i,j Ø Ø	0,1378	0,3483
A,B,D,E,G,I,J C F H a,b,c,d,e,f,g,h,i Ø Ø Ø	0,0656	0,2249
A B C,D,E,G,H,I,J F Ø b,c,d,e,f,g,h,i,j Ø	0,1610	0,2510
A,B,E,F,G,I,J C D H b,c,d,e,f,g,h,i Ø Ø Ø	0,0985	0,1754
A C,D,E,F,G,I,J B H Ø a,b,d,e,g,h,j Ø Ø	0,0874	0,1370
A,B,C,E,H,I,J D F G a,c,e,g,i Ø Ø Ø	0,0161	0,0970
A,D,F,G,H,I,J B C E b,d,f,h,j Ø Ø Ø	0,0283	0,0905
A C,D,E,G,H,I B F Ø a,b,c,d,e,f,g,h,i,j Ø Ø	0,1378	0,2100
A,B,D,E,G,I C F H a,b,c,d,e,f,g,h,i Ø Ø Ø	0,0656	0,1380
A B C,D,E,G,H,I F Ø b,c,d,e,f,g,h,i,j Ø	0,1610	0,2248
A,B,E,F,G,I C D H b,c,d,e,f,g,h,i Ø Ø Ø	0,0985	0,2367
A C,D,E,F,G,I B H Ø a,b,d,e,g,h,j Ø Ø	0,0874	0,1359
A,B,C,E,H,I D F G a,c,e,g,i Ø Ø Ø	0,0161	0,0859
A,D,F,G,H,I B C E b,d,f,h,j Ø Ø Ø	0,0283	0,0876
C,D,E,G,H,I B F a,b,c,d,e,f,g,h,i,j Ø Ø Ø	0,1995	0,2345
B,D,E,G,I,J C F H a,b,c,d,e,f,g,h,i Ø Ø Ø	0,0656	0,2065
B D,E,G,H,I,J C F Ø b,c,d,e,f,g,h,i,j Ø Ø	0,1971	0,2339
B,E,F,G,I,J C D H b,c,d,e,f,g,h,i Ø Ø Ø	0,0985	0,1645
C,D,E,F,G,I B H a,b,d,e,g,h,j Ø Ø Ø	0,0932	0,1248
B,C,E,H,I,J D F G a,c,e,g,i Ø Ø Ø	0,0161	0,0833
B D,E,G,H,I,I C E Ø b,d,f,h,i Ø Ø	0,0283	0,0848

7. Bibliografía

1. Visual Studio Code y Visual Studio Tools: [Visual Studio: IDE y editor de código para el desarrollo de software](#)
2. Python: [Welcome to Python.org](#)
3. Git: [Git](#)
4. Guías del Proyecto